

DevOps Lab L3 Assignment Submission TW2

Name: Sanyukt Ramola

PNR: 23070122194

GitHub Repo: <https://github.com/Kranvagn4/hello-world-flask>

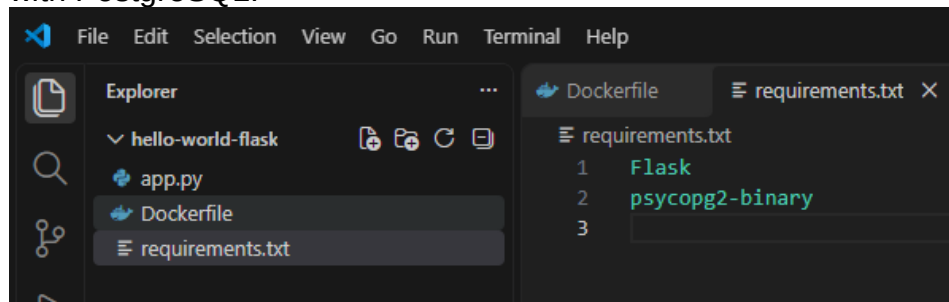
Part 1: Docker Compose and PostgreSQL:

Task 1.1: Modify Flask to use PostgreSQL:

Make sure that docker is running and stop the Flask server running previously.

```
sanyu@SANYUKT MINGW64 ~  
$ cd ~/OneDrive/Desktop/Studies/hello-world-flask  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)  
$ pwd  
/c:/Users/sanyu/OneDrive/Desktop/Studies/hello-world-flask  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)  
$
```

Modify requirements.txt to add psycopg2 Python library, which allows the Py app to communicate with PostgreSQL:

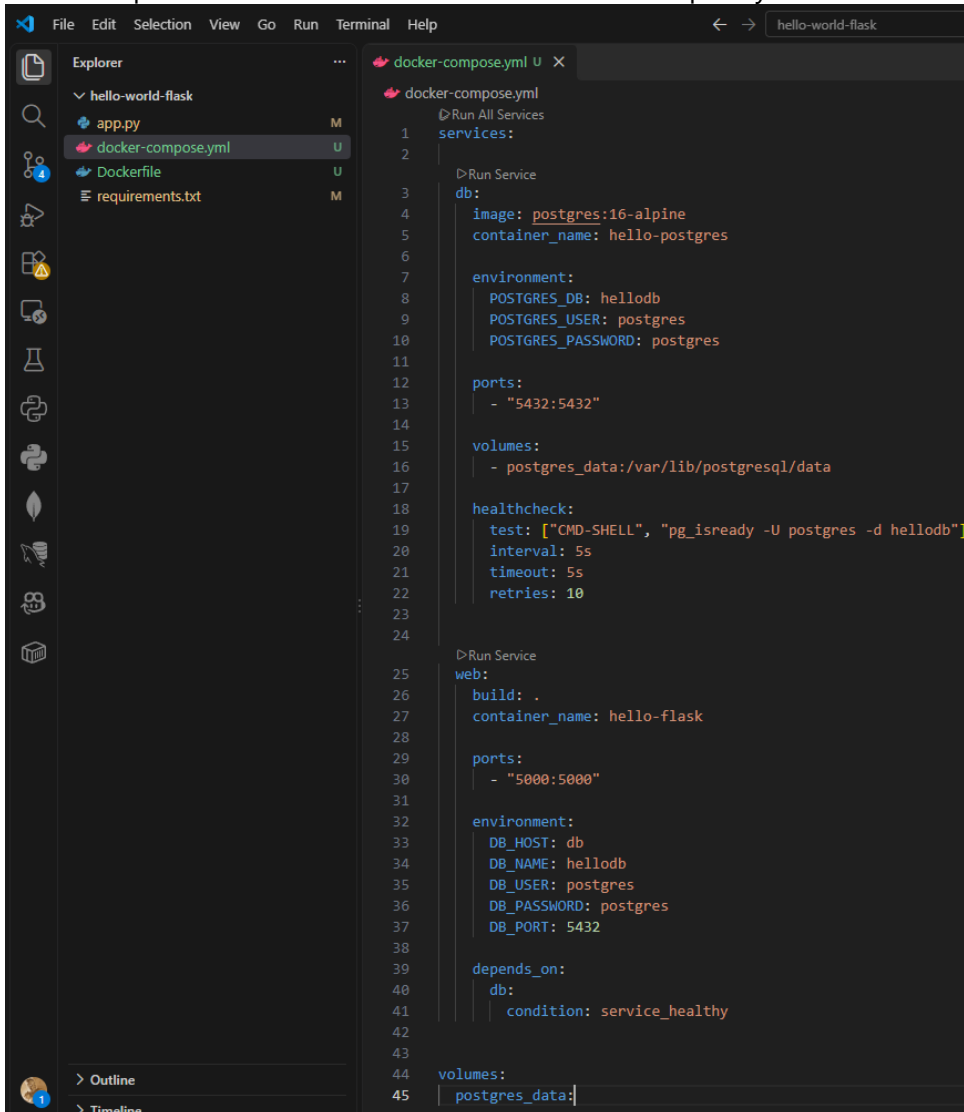


Now, we need to update our old app.py to fit the requirements.txt:

```
app.py 1, M X  
app.py > get_db_connection  
1 from flask import Flask  
2 import psycopg2  
3 import os  
4 import time  
5  
6 app = Flask(__name__)  
7  
8  
9 def get_db_connection():  
10     retries = 10  
11  
12     while retries > 0:  
13         try:  
14             connection = psycopg2.connect(  
15                 host=os.getenv("DB_HOST", "localhost"),  
16                 database=os.getenv("DB_NAME", "hellobd"),  
17                 user=os.getenv("DB_USER", "postgres"),  
18                 password=os.getenv("DB_PASSWORD", "postgres"),  
19                 port=os.getenv("DB_PORT", "5432")  
20             )  
21             return connection  
22  
23         except psycopg2.OperationalError:  
24             retries -= 1  
25             print("Waiting for PostgreSQL...")  
26             time.sleep(2)  
27  
28     raise Exception("Could not connect to PostgreSQL")  
29  
30  
31  
32 @app.route("/")  
33 def hello():  
34     connection = get_db_connection()  
35     cursor = connection.cursor()  
36  
37     cursor.execute("""  
38         CREATE TABLE IF NOT EXISTS messages (  
39             id SERIAL PRIMARY KEY,  
40             message TEXT NOT NULL  
41         )  
42     """)  
43  
44     cursor.execute(  
45         "INSERT INTO messages (message) VALUES (%s)",  
46         ("Hello World from PostgreSQL!",)  
47     )  
48  
49     connection.commit()  
50  
51     cursor.execute(  
52         "SELECT message FROM messages ORDER BY id DESC LIMIT 1"  
53     )  
54  
55     message = cursor.fetchone()[0]  
56  
57     cursor.close()  
58     connection.close()  
59  
60     return message  
61  
62  
63 if __name__ == "__main__":  
64     app.run(host="0.0.0.0", port=5000)
```

Task 1.2: Creation of Docker-compose.yml and Running the app:

We then update the Dockerfile and create "docker-compose.yml"



The screenshot shows the Visual Studio Code editor with a project named 'hello-world-flask'. The Explorer sidebar on the left shows files: 'app.py', 'docker-compose.yml' (selected), 'Dockerfile', and 'requirements.txt'. The main editor displays the 'docker-compose.yml' file with the following content:

```
1 services:
2
3   >Run Service
4   db:
5     image: postgres:16-alpine
6     container_name: hello-postgres
7
8     environment:
9       POSTGRES_DB: helldb
10      POSTGRES_USER: postgres
11      POSTGRES_PASSWORD: postgres
12
13     ports:
14       - "5432:5432"
15
16     volumes:
17       - postgres_data:/var/lib/postgresql/data
18
19     healthcheck:
20       test: ["CMD-SHELL", "pg_isready -U postgres -d helldb"]
21       interval: 5s
22       timeout: 5s
23       retries: 10
24
25   >Run Service
26   web:
27     build: .
28     container_name: hello-flask
29
30     ports:
31       - "5000:5000"
32
33     environment:
34       DB_HOST: db
35       DB_NAME: helldb
36       DB_USER: postgres
37       DB_PASSWORD: postgres
38       DB_PORT: 5432
39
40     depends_on:
41       db:
42         condition: service_healthy
43
44   volumes:
45     postgres_data:
```

Check docker version in Git to ensure consistency:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ docker compose version
Docker Compose version v5.1.1

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
```

Now, start the docker compose build:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ docker compose up --build
Image hello-world-flask-web Building
#1 [internal] load local bake definitions
#1 reading from stdin 587B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 205B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 1.1s

#4 [internal] load .dockerignore
#4 transferring context: 2B 0.0s done
#4 DONE 0.0s

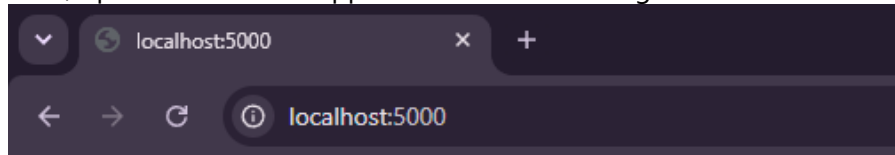
#5 [internal] load build context
#5 transferring context: 63B done
#5 DONE 0.0s
```

```

hello-postgres | 2026-09-30 17:07:26.351 UTC [1] LOG: database system is ready to accept connections
Container hello-postgres Healthy
Container hello-flask Starting
Container hello-flask Started
hello-flask | * Serving Flask app 'app'
hello-flask | * Debug mode: off
hello-flask | WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
hello-flask | * Running on all addresses (0.0.0.0)
hello-flask | * Running on http://127.0.0.1:5000
hello-flask | * Running on http://172.19.0.3:5000
hello-flask | Press CTRL+C to quit

```

Now, open the localhost application to see th changes:



Hello World from PostgreSQL!

We sucessfully connected our Flask application to PostgreSQL and ran its Docker build.

Now, check the container through another Bash shell:

```

MINGW64:/c:/Users/sanyu/OneDrive/Desktop/Studies/hello-world-flask
sanyu@SANYUKT MINGW64 ~
$ cd ~/OneDrive/Desktop/Studies/hello-world-flask

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ docker compose ps
NAME                IMAGE                        COMMAND                  SERVICE    CREATED        STATUS
hello-flask         hello-world-flask-web       "python app.py"         web        5 minutes ago  Up 4 minutes
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
hello-postgres      postgres:16-alpine          "docker-entrypoint.s..." db         9 minutes ago  Up 5 minutes (healthy)
0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$

```

We see both the flask app and the postgres running healthy.

Now, we check the database directly:

```

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ winpty docker exec -it hello-postgres psql -U postgres -d hellodb
psql (16.15)
Type "help" for help.

hellodb=# SELECT * FROM messages;
 id |          message
-----+-----
  1 | Hello World from PostgreSQL!
(1 row)

hellodb=#

```

Hence, our Flask app ran sucessfully.

Part 2: Kubernetes Deployment and Service:

Task 2.1: Docker image accessibility by Minikube:

Since I'll be working on this project locally on Windows, I'll be using Minikube:

```
sanyu@SANYUKT MINGW64 ~
$ minikube version
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10

sanyu@SANYUKT MINGW64 ~
$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1

sanyu@SANYUKT MINGW64 ~
$ cd ~/OneDrive/Desktop/Studies/hello-world-flask

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
```

Starting Minikube:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ minikube start --driver=docker
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
  > gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 10.23 MiB
d
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.34.1, which may have incompatibilities with Kubernetes 1.37.0.
  - Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Now, we use Minikube's image loading capability instead of pushing it to the Docker hub:

First, we build the new image:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ docker build -t hello-world-flask:tw2 .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 205B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 2.0s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 63B done
#4 DONE 0.0s

#5 [1/5] FROM docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4930f79523f93f81bbc7632584ec65e36ab055f41a800b41e
#5 resolve docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4930f79523f93f81bbc7632584ec65e36ab055f41a800b41e 0.0s done
#5 DONE 0.0s

#6 [3/5] COPY requirements.txt .
#6 CACHED

#7 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#7 CACHED

#8 [2/5] WORKDIR /app
#8 CACHED

#9 [5/5] COPY app.py .
#9 CACHED

#10 exporting to image
#10 exporting layers done
#10 exporting manifest sha256:80890a36cbe523147588a33042ec69c50e8b397fdc98a07fd8fd87e0adb35e0a 0.0s done
#10 exporting config sha256:e9b87a488e8377b56fe38bf251e92b363cf183635235a7857a97ad22a1ffe645 done
#10 exporting attestation manifest sha256:716470d44fcb079cf56f49b361d2d66ef1085dac632ed8c85ac5f92d9821a1e1 0.0s done
#10 exporting manifest list sha256:eb345c9dbef07c6c63648806daba1e7d61136dc78a497fabfadbc49bbb562fb1 done
#10 naming to docker.io/library/hello-world-flask:tw2
#10 naming to docker.io/library/hello-world-flask:tw2 done
#10 unpacking to docker.io/library/hello-world-flask:tw2 done
#10 DONE 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/k5u2md5qku19ognraktct2jax
```

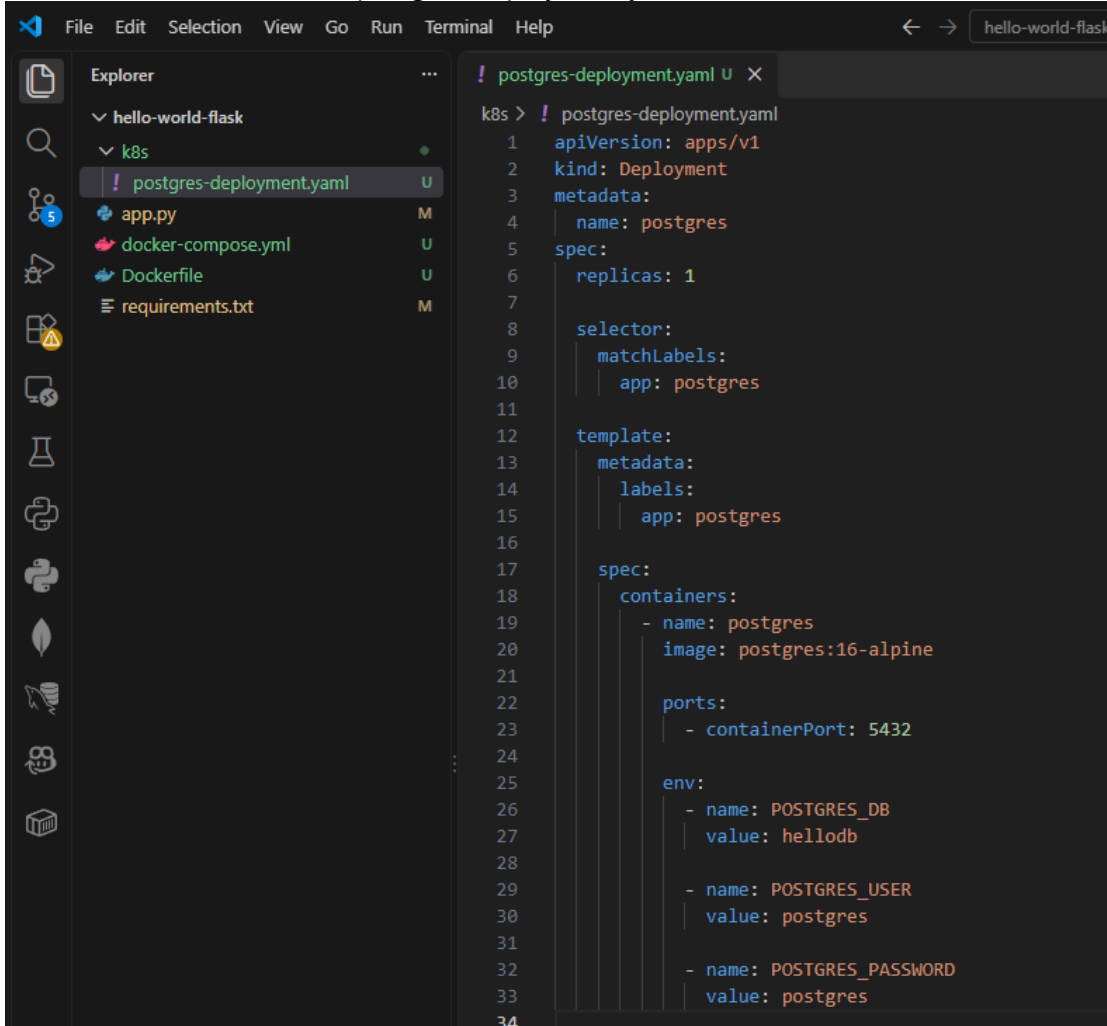
Then, we load it into minikube:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ minikube image load hello-world-flask:tw2

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
```

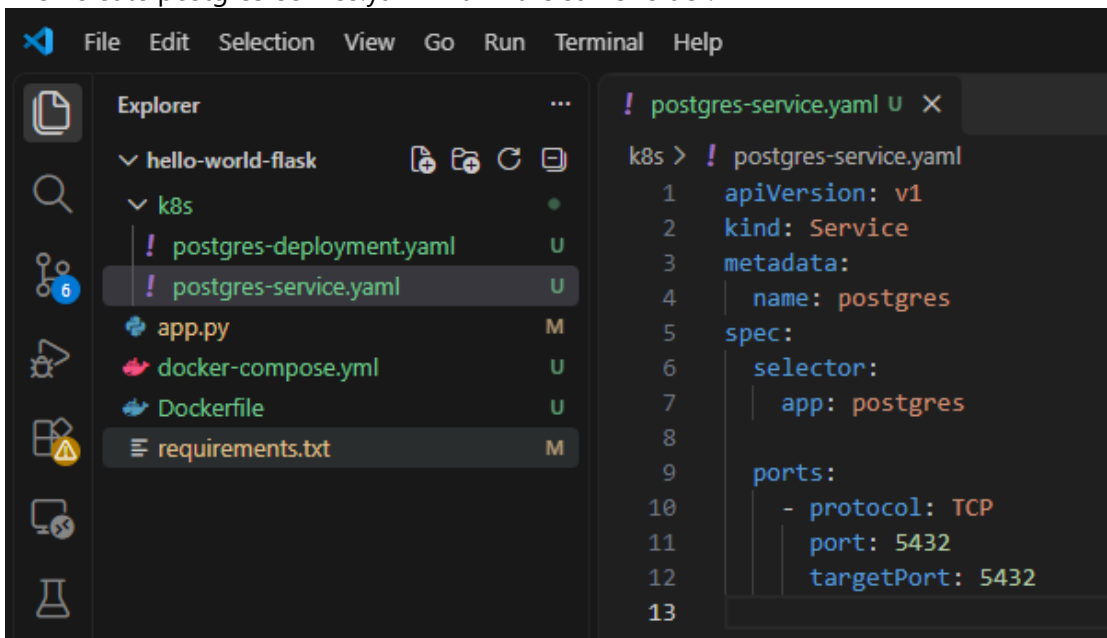
Next, we create the Kubernetes folder.

Inside this k8s folder, create postgres-deployment.yaml:

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project named 'hello-world-flask' with a subfolder 'k8s'. Inside 'k8s', the file 'postgres-deployment.yaml' is selected. The main editor area displays the content of this file. The file is a Kubernetes Deployment manifest for a PostgreSQL service. It includes fields for apiVersion, kind, metadata (name), spec (replicas), selector (matchLabels), template (metadata labels), and spec (containers). The container is named 'postgres' and uses the 'postgres:16-alpine' image. It also defines environment variables for database connection details.

```
k8s > ! postgres-deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: postgres
5  spec:
6    replicas: 1
7
8    selector:
9      matchLabels:
10       app: postgres
11
12    template:
13      metadata:
14        labels:
15          app: postgres
16
17      spec:
18        containers:
19          - name: postgres
20            image: postgres:16-alpine
21
22            ports:
23              - containerPort: 5432
24
25            env:
26              - name: POSTGRES_DB
27                value: hellobdb
28
29              - name: POSTGRES_USER
30                value: postgres
31
32              - name: POSTGRES_PASSWORD
33                value: postgres
34
```

Then create postgres-service.yaml within the same folder:

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows the 'k8s' folder with two files: 'postgres-deployment.yaml' and 'postgres-service.yaml'. The 'postgres-service.yaml' file is selected. The main editor area displays the content of this file. It is a Kubernetes Service manifest for the 'postgres' service. It includes fields for apiVersion, kind, metadata (name), spec (selector), and spec (ports). The selector matches the 'app: postgres' label. The port configuration shows a protocol of TCP, a port of 5432, and a targetPort of 5432.

```
k8s > ! postgres-service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: postgres
5  spec:
6    selector:
7      app: postgres
8
9    ports:
10     - protocol: TCP
11       port: 5432
12       targetPort: 5432
13
```

This yaml file gives our Flask app pod a hostname: postgres, which allowed it to be connected via

"DB_HOST=postgres" command
Then we deploy PostgreSQL in Bash:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl apply -f k8s/postgres-deployment.yaml
deployment.apps/postgres created

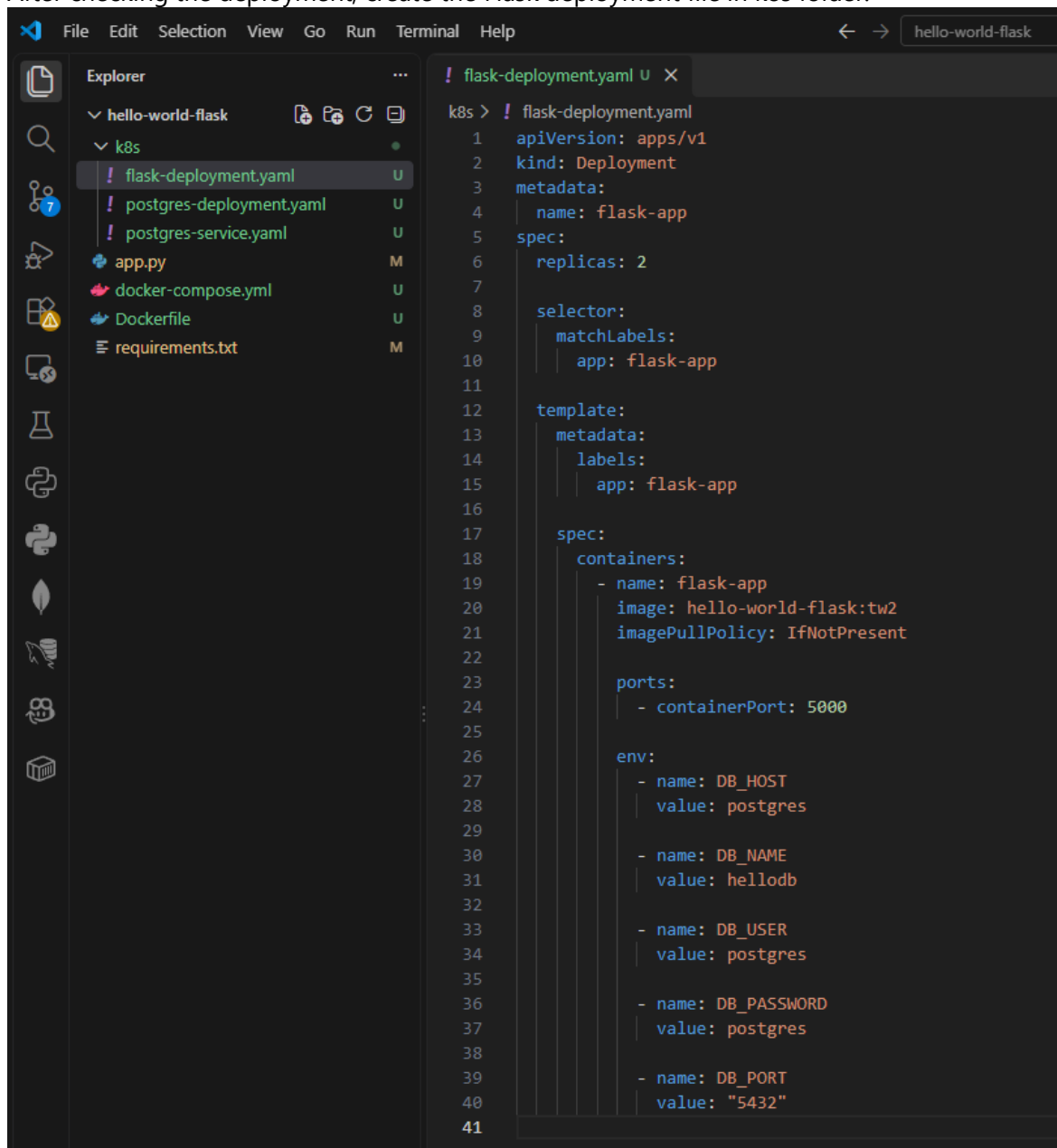
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl apply -f k8s/postgres-service.yaml
service/postgres created

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
postgres-5dc9d4cdcc-whsp7          0/1     ContainerCreating   0           15s

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl get services
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes  ClusterIP   10.96.0.1    <none>        443/TCP    11m
postgres   ClusterIP   10.106.93.91 <none>        5432/TCP   41s
```

Task 2.2: Kubernetes Deployment for Flask Application:

After checking the deployment, create the Flask deployment file in k8s folder:



The screenshot shows a Visual Studio Code editor with the 'hello-world-flask' project open. The Explorer sidebar on the left shows the file structure, with the 'k8s' folder expanded and 'flask-deployment.yaml' selected. The main editor displays the content of 'flask-deployment.yaml'.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: flask-app
5 spec:
6   replicas: 2
7
8   selector:
9     matchLabels:
10      app: flask-app
11
12   template:
13     metadata:
14       labels:
15        app: flask-app
16
17     spec:
18       containers:
19         - name: flask-app
20           image: hello-world-flask:tw2
21           imagePullPolicy: IfNotPresent
22
23       ports:
24         - containerPort: 5000
25
26       env:
27         - name: DB_HOST
28           value: postgres
29
30         - name: DB_NAME
31           value: hellobdb
32
33         - name: DB_USER
34           value: postgres
35
36         - name: DB_PASSWORD
37           value: postgres
38
39         - name: DB_PORT
40           value: "5432"
41
```

Here, the replicas: 2 is done as it creates two Flask pods (Pod 1 and 2), which demonstrates Kubernetes managing multiple application instances concurrently.

Flask Deployment:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl apply -f k8s/flask-deployment.yaml
deployment.apps/flask-app created

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-6978db864-cvfrw           1/1     Running   0           10s
flask-app-6978db864-j58s7           1/1     Running   0           10s
postgres-5dc9d4cdcc-whsp7          1/1     Running   0           4m10s

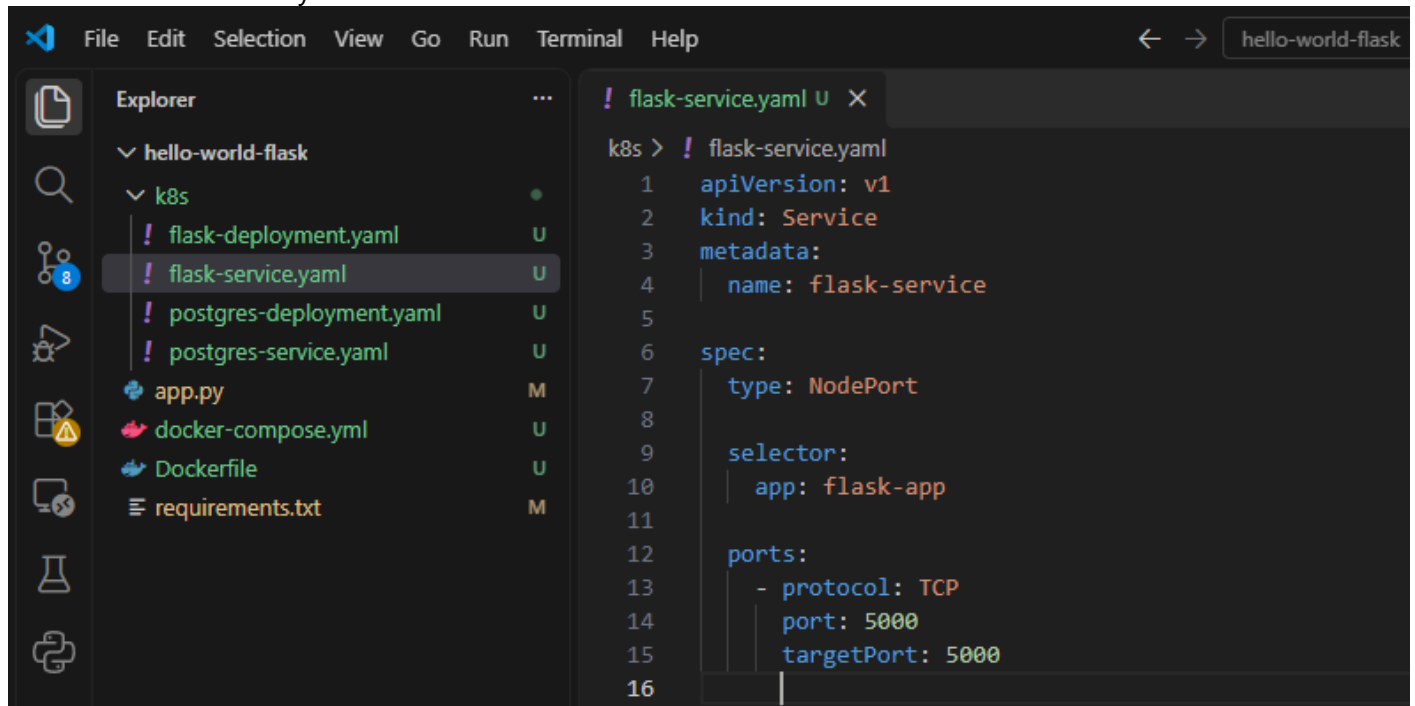
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
flask-app 2/2      2             2           33s
postgres  1/1      1             1           4m33s

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
```

Task 2.3: Kubernetes Service:

Create file flask-service.yaml inside the k8s folder:



The screenshot shows the Visual Studio Code editor with the file `flask-service.yaml` open. The file content is as follows:

```
k8s > ! flask-service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: flask-service
5
6  spec:
7    type: NodePort
8
9    selector:
10     app: flask-app
11
12    ports:
13     - protocol: TCP
14       port: 5000
15       targetPort: 5000
16
```

Apply the Kubernetes service:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl apply -f k8s/flask-service.yaml
service/flask-service created

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ kubectl get services
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
flask-service   NodePort    10.108.14.93 <none>        5000:31398/TCP   5s
kubernetes      ClusterIP   10.96.0.1    <none>        443/TCP          22m
postgres        ClusterIP   10.106.93.91 <none>        5432/TCP         11m

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
```

We can see the newly created Flask service.

Now, we can access Flask using Bash:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ minikube service flask-service
```

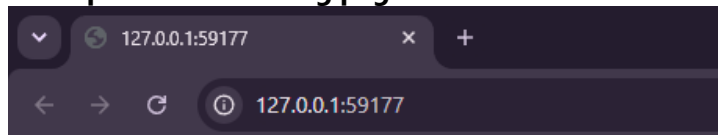
NAMESPACE	NAME	TARGET PORT	URL
default	flask-service	5000	http://192.168.49.2:31398

```
* Starting tunnel for service flask-service.
```

NAMESPACE	NAME	TARGET PORT	URL
default	flask-service		http://127.0.0.1:59177

```
* Opening service default/flask-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

This opens the following page:



Hello World from PostgreSQL!

Stop the service:

```
* Stopping tunnel for service flask-service.
```

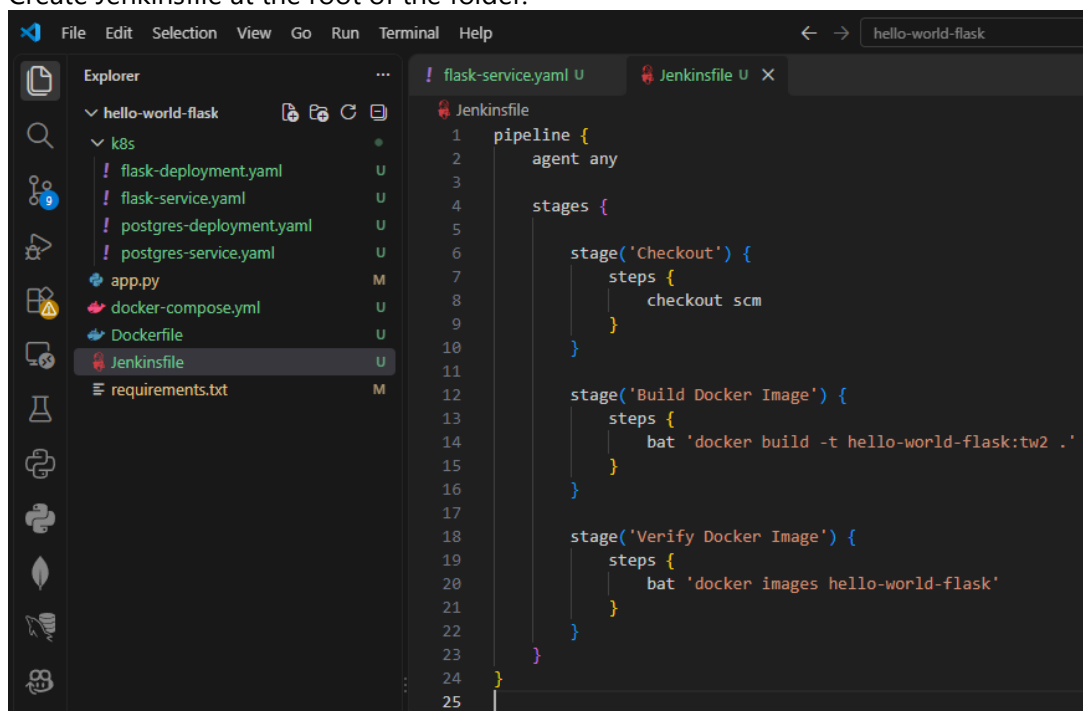
```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ |
```

Hence, the Kubernetes Service successfully loaded the Flask application, and Flask is communicating with PostgreSQL and launching its Pod.

Part 3: Integrated CI/CD with Jenkins and Infrastructure as Code:

Task 3.1: Create Jenkins Declarative Pipeline:

Create Jenkinsfile at the root of the folder:



This file gets the code from GitHub :D

We then push this Jenkinsfile into our GitHub repository:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git add Jenkinsfile

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git commit -m "Add Jenkins declarative pipeline"
[main b24b5a0] Add Jenkins declarative pipeline
1 file changed, 24 insertions(+)
create mode 100644 Jenkinsfile

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 489 bytes | 489.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Kranvagn4/hello-world-flask
1ba7cff..b24b5a0 main -> main

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
```

Next, we create the Jenkins pipeline project:

New Item

Enter an item name

Hello-World-Flask-Pipeline

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Jenkins / Hello-World-Flask-Pipeline / Configure

Configure

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Kranvagn4/hello-world-flask.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Save Apply

Add the script path at the end as Jenkinsfile:

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

Pipeline Syntax

Before we build the pipeline, lets add and commit all the updated files to the GitHub repository:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git add app.py requirements.txt docker-compose.yml k8s/

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   app.py
    new file:   docker-compose.yml
    new file:   k8s/flask-deployment.yml
    new file:   k8s/flask-service.yml
    new file:   k8s/postgres-deployment.yml
    new file:   k8s/postgres-service.yml
    modified:   requirements.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Dockerfile

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git commit -m "Add Docker Compose and Kubernetes configuration"
[main 71d3e60] Add Docker Compose and Kubernetes configuration
 7 files changed, 207 insertions(+), 5 deletions(-)
 create mode 100644 docker-compose.yml
 create mode 100644 k8s/flask-deployment.yml
 create mode 100644 k8s/flask-service.yml
 create mode 100644 k8s/postgres-deployment.yml
 create mode 100644 k8s/postgres-service.yml

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 28 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.19 KiB | 2.19 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Kranvagn4/hello-world-flask
   b24b5a0..71d3e60  main -> main

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git add Dockerfile

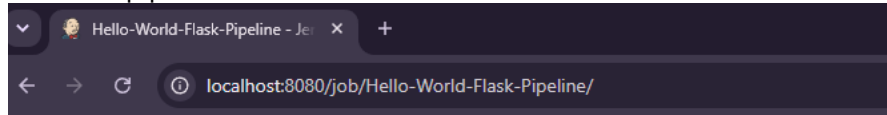
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   Dockerfile

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git commit -m "Add Dockerfile for TW2"
[main c1dc959] Add Dockerfile for TW2
 1 file changed, 7 insertions(+)
 create mode 100644 Dockerfile

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 418 bytes | 418.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Kranvagn4/hello-world-flask
   71d3e60..c1dc959  main -> main
```

Build the pipeline:



Jenkins / Hello-World-Flask-Pipeline

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Stages

Pipeline Syntax

✓ Hello-World-Flask-Pipeline

Permalinks

- Last build (#2), 21 sec ago
- Last stable build (#2), 21 sec ago
- Last successful build (#2), 21 sec ago
- Last completed build (#2), 21 sec ago

Builds >

Filter

Today

✓ #2 11:43 PM

✓ #2 (Sep 30, 2026, 11:43:06 PM)



Started by user [Sanyukt Ramola](#)



Revision: c1dc959209abb0d9c48929bd27a4bd13f921792c

Repository: <https://github.com/Kranvagn4/hello-world-flask.git>

- refs/remotes/origin/main



No changes.

Console Output:

localhost:8080/job/Hello-World-Flask-Pipeline/2/console

Jenkins / Hello-World-Flask-Pipeline / #2

Status

Changes

Console Output

Delete build #2

Git Build Data

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Console

Download

Started by user [Sanyukt Ramola](#)

Obtained Jenkinsfile from git <https://github.com/Kranvagn4/hello-world-flask.git>

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\Hello-World-Flask-Pipeline

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Declarative: Checkout SCM)

[Pipeline] checkout

Selected Git installation does not exist. Using Default

The recommended git tool is: NONE

No credentials specified

> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Hello-World-Flask-Pipeline\.git # timeout=10

Fetching changes from the remote Git repository

> git.exe config remote.origin.url <https://github.com/Kranvagn4/hello-world-flask.git> # timeout=10

Fetching upstream changes from <https://github.com/Kranvagn4/hello-world-flask.git>

> git.exe --version # timeout=10

> git --version # 'git version 2.46.0.windows.1'

> git.exe fetch --tags --force --progress -- <https://github.com/Kranvagn4/hello-world-flask.git> +refs/heads/*:refs/remotes/origin/* # timeout=10

> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10

Checking out Revision c1dc959209abb0d9c48929bd27a4bd13f921792c (refs/remotes/origin/main)

> git.exe config core.sparsecheckout # timeout=10

> git.exe checkout -f c1dc959209abb0d9c48929bd27a4bd13f921792c # timeout=10

Commit message: "Add Dockerfile for TN2"

First time build. Skipping changelog.

```

[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\Hello-World-Flask-Pipeline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/Kranvagn4/hello-world-flask.git # timeout=10
Fetching upstream changes from https://github.com/Kranvagn4/hello-world-flask.git
> git.exe --version # timeout=10
> git --version # 'git version 2.46.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/Kranvagn4/hello-world-flask.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision c1dc959209abb0d9c48929bd27a4bd13f921792c (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f c1dc959209abb0d9c48929bd27a4bd13f921792c # timeout=10
Commit message: "Add Dockerfile for TW2"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Hello-World-Flask-Pipeline>docker build -t hello-world-flask:tw2 .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 205B done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 1.8s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [1/5] FROM docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4930f79523f93f81bbc7632584ec65e36ab055f41a800b41e
#4 resolve docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4930f79523f93f81bbc7632584ec65e36ab055f41a800b41e 0.0s done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 1.56kB 0.0s done
#5 DONE 0.0s

#6 [2/5] WORKDIR /app
#6 CACHED

#7 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#7 CACHED

#8 [3/5] COPY requirements.txt .
#8 CACHED

#9 [5/5] COPY app.py .
#9 CACHED

#10 exporting to image
#10 exporting layers done
#10 exporting manifest sha256:80890a36cbe523147588a33042ec69c50e8b397fdc98a07fd8fd87e0adb35e0a done
#10 exporting config sha256:e9b87a488e8377b56fe38bf251e92b363cf183635235a7857a97ad22a1ffe645 done
#10 exporting attestation manifest sha256:a988b2247ac5c58ecc90f4e394089ef4aa19b1e582eca4c9fd5e8105225ec7 0.0s done
#10 exporting manifest list sha256:b5fd28aa130b389273f2e551da6c6daab0ae8ced5ebc6e8be476f3635167c2b1
#10 exporting manifest list sha256:b5fd28aa130b389273f2e551da6c6daab0ae8ced5ebc6e8be476f3635167c2b1 done
#10 naming to docker.io/library/hello-world-flask:tw2 done
#10 unpacking to docker.io/library/hello-world-flask:tw2 done
#10 DONE 0.1s

```

```

[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Docker Image)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Hello-World-Flask-Pipeline>docker images hello-world-flask
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
hello-world-flask:latest a962b131e1fe      211MB       51.3MB
hello-world-flask:tw2  b5fd28aa130b      228MB       55.8MB
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Hence, Jenkins has successfully built the Docker image, Verified the Docker image and gave the console output as a success.

Updated GitHub repository after task 3.1:

The screenshot displays the GitHub interface for the repository 'hello-world-flask' owned by 'Kranvagn4'. The repository is public. The file list shows several files added recently, including 'k8s', 'Dockerfile', 'Jenkinsfile', 'app.py', 'docker-compose.yml', and 'requirements.txt'. The README section is currently empty, with a prompt to 'Add a README'. The right sidebar shows repository statistics: 0 stars, 0 forks, and 0 releases. The languages section shows Python at 89.7% and Dockerfile at 10.3%.

File	Description	Time
k8s	Add Docker Compose and Kubernetes...	9 minutes ago
Dockerfile	Add Dockerfile for TW2	8 minutes ago
Jenkinsfile	Add Jenkins declarative pipeline	21 minutes ago
app.py	Add Docker Compose and Kubernetes...	9 minutes ago
docker-compose.yml	Add Docker Compose and Kubernetes...	9 minutes ago
requirements.txt	Add Docker Compose and Kubernetes...	9 minutes ago

README

Add a README

Help people interested in this repository understand your project.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

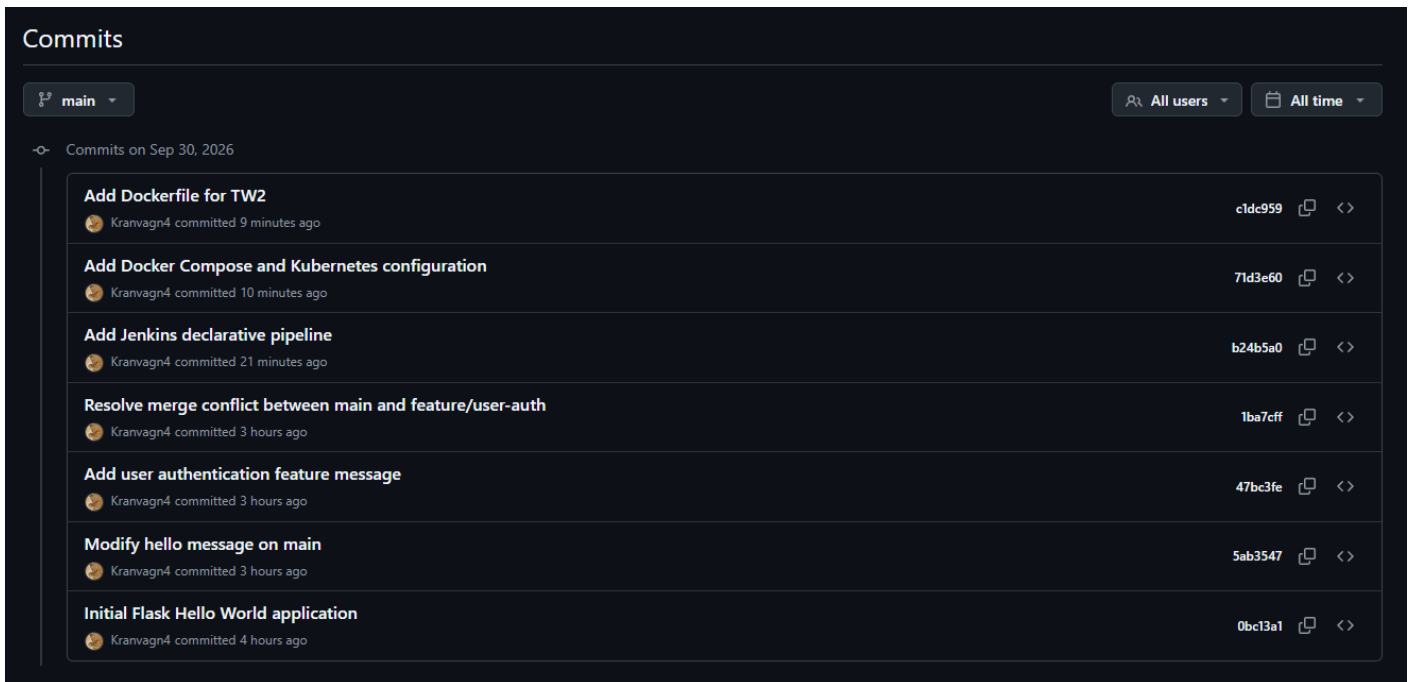
[Publish your first package](#)

Contributors 1

Kranvagn4 Kronovi^_^

Languages

Python 89.7% Dockerfile 10.3%



Task 3.2: Terraform:

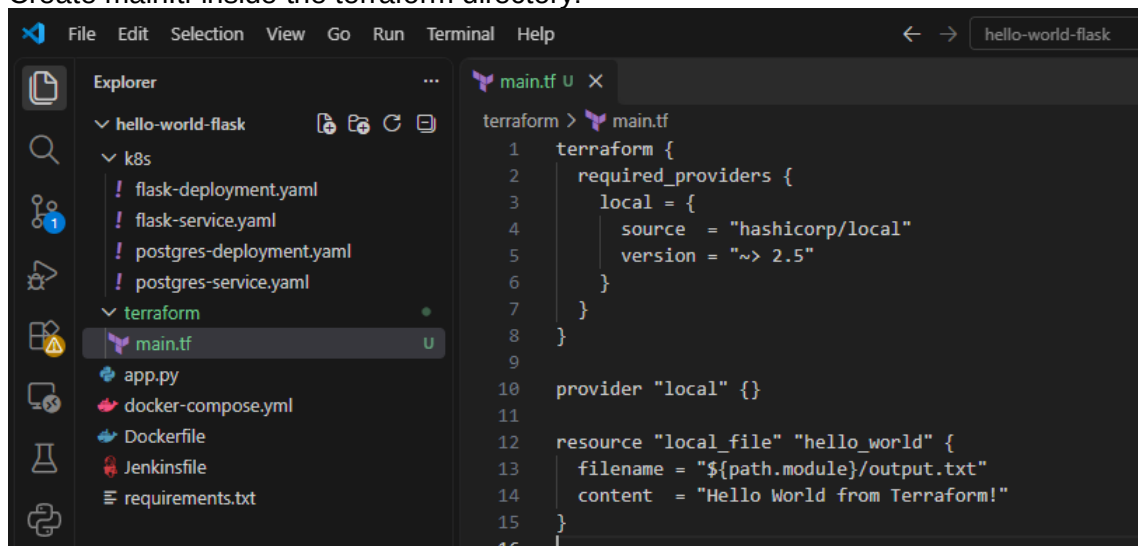
Install terraform, add its exe file folder's path into system environment variables, and then check its version:

```
sanyu@SANYUKT MINGW64 ~  
$ terraform --version  
Terraform v1.16.4  
on windows_amd64
```

Add terraform directory into the root project folder:

```
sanyu@SANYUKT MINGW64 ~  
$ cd ~/OneDrive/Desktop/Studies/hello-world-flask  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)  
$ ls  
Dockerfile Jenkinsfile app.py docker-compose.yml k8s/ requirements.txt  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)  
$ mkdir terraform  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)  
$ cd terraform  
  
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)  
$
```

Create main.tf inside the terraform directory:



Terraform creates output.txt with the output as "Hello World from Terraform!"

Initialize terraform inside the terraform directory:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask (main)
$ cd terraform

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ |
```

Run terraform plan:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ terraform plan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.hello_world will be created
+ resource "local_file" "hello_world" {
  + content           = "Hello World from Terraform!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "./output.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ |
```

The terraform plan tells us what the terraform

Apply the terraform:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ terraform apply

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.hello_world will be created
+ resource "local_file" "hello_world" {
  + content          = "Hello World from Terraform!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename         = "./output.txt"
  + id               = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.hello_world: Creating...
local_file.hello_world: Creation complete after 0s [id=ce53dc23b86bcd60b57493385905d90312fa85bc]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$
```

Check the files:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ ls
main.tf  output.txt  terraform.tfstate

sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$
```

Use cat function on output.txt to view the output:

```
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ cat output.txt
Hello World from Terraform!
sanyu@SANYUKT MINGW64 ~/OneDrive/Desktop/Studies/hello-world-flask/terraform (main)
$ |
```

Hence, terraform has successfully created the output file and displayed the message "Hello World!" in it.